

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER ARCHITECTURE COURSE CODE: CSA12

COURSE OUTCOME COVERED

CO1: Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking(BL4)

Assessment Tool Weightage: 40%

QUESTIONS:5

Total Marks:25 ¹⁰⁰

ANNEXURE A APPLICATION BASED QUESTIONS

83%

Code of Conduct:

I Kailash sharma certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Kailash
Signature of the student:

ANNEXURE A

1. A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.

Problem:

- A smart city surveillance system continuously receives video streams from multiple cameras.
 - the number of video streams increases significantly, causing delays in threat detection
- the CPU, memory, and I/O devices compete for system resources.

Application of Concepts in a Real-Time Smart City Surveillance System

CPU, Memory, and I/O Interaction:

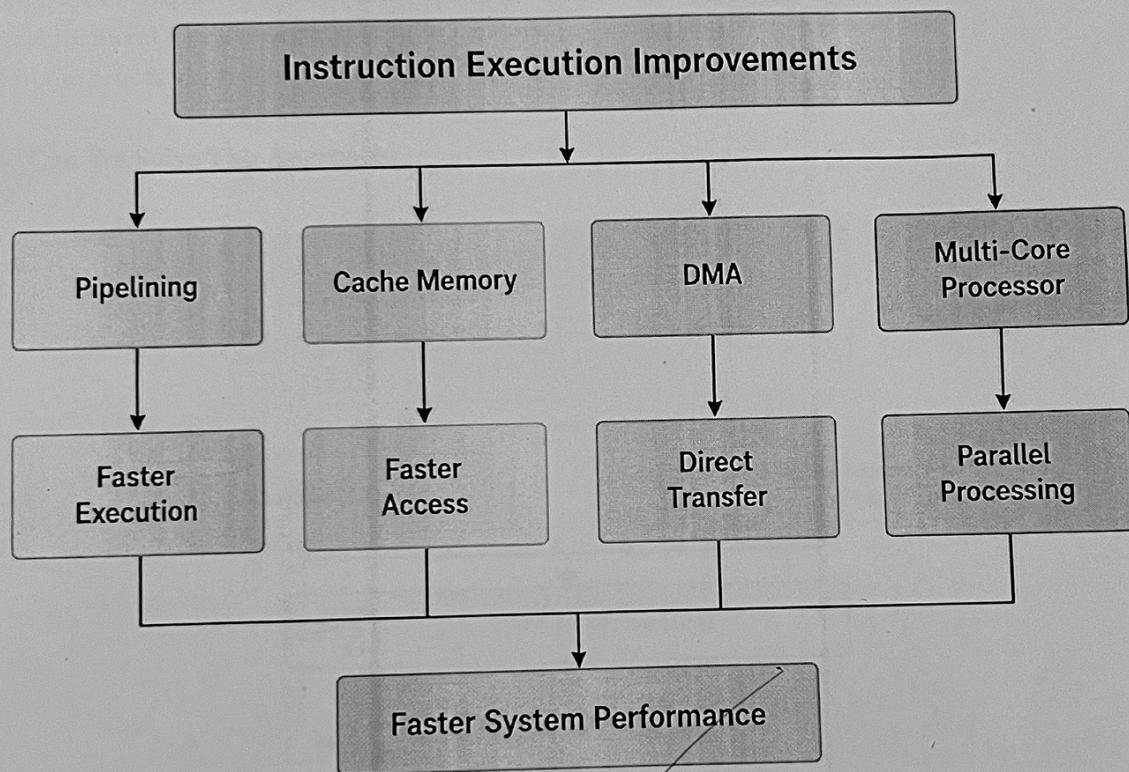
- CCTV cameras (I/O devices) send live video to RAM, where the CPU retrieves and processes the data to detect threats.
- During peak hours, heavy data flow increases CPU and memory workload, causing delays.

Application of Bus structure:

A **single-bus** system creates congestion because CPU, memory, and I/O share one communication path.

A **multi-bus** system allows simultaneous data transfers, reducing bottlenecks and improving real-time performance.

Improvements:



- ❑ **Instruction Pipelining** – Processes multiple instructions at the same time.
- ❑ **Cache Memory** – Stores frequently used data for faster CPU access.
- ❑ **DMA** – Transfers data directly from I/O devices to RAM without using the CPU all the time.
- ❑ **Multi-Core Processor** – Uses multiple CPU cores to process several tasks simultaneously.

In the smart city surveillance system, these improvements help the CPU analyze video data faster, resulting in quicker threat detection and better system performance.

Conclusion: These improvements make the surveillance system faster and reduce processing delays.

2. A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch-decode-execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.

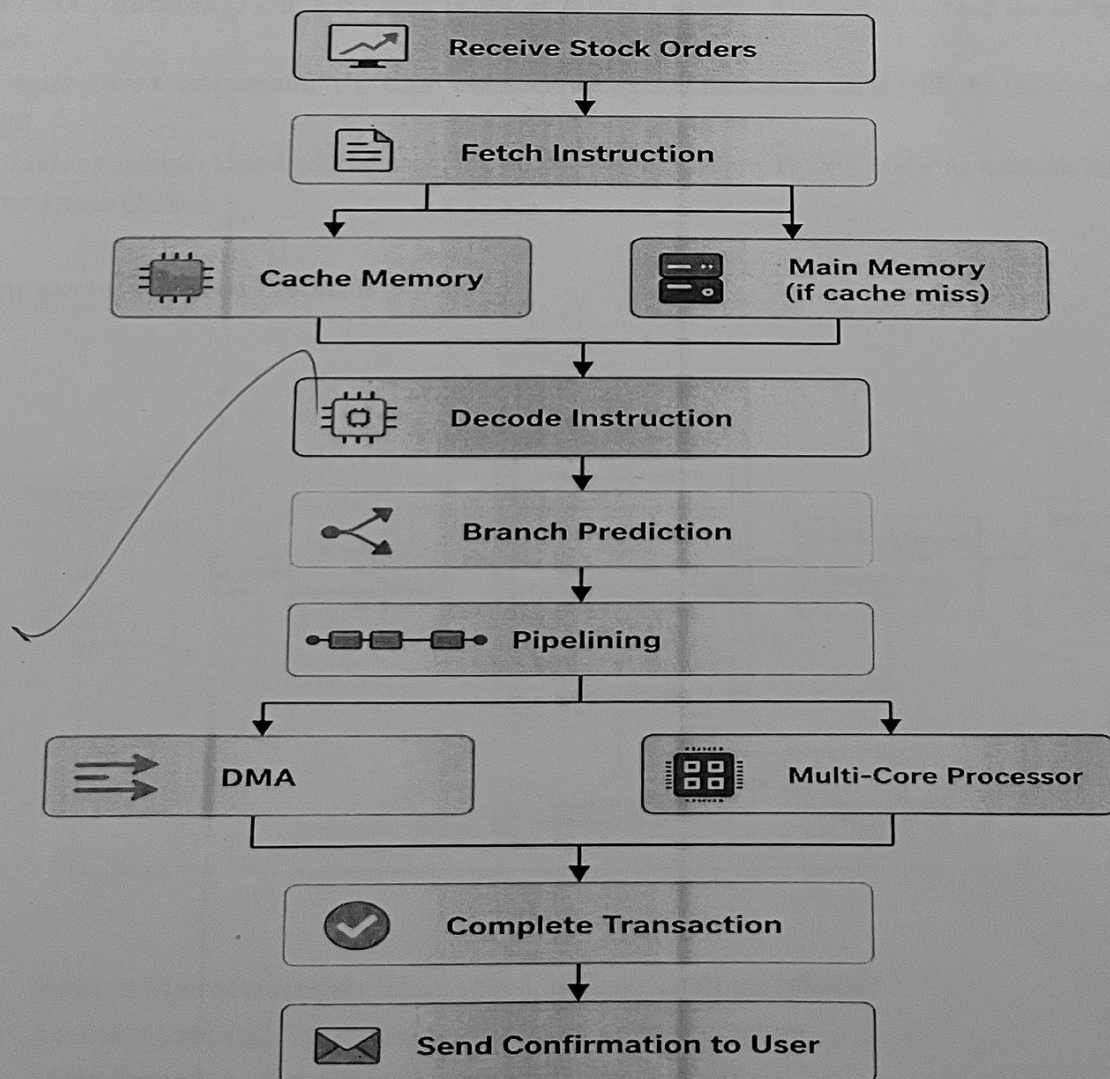
Understanding of the Problem Context:

A real-time stock trading application handles millions of transactions every second. During heavy trading, frequent memory access and branch instructions slow down the processor, causing delays.

Application:

- **Fetch-Decode-Execute Cycle** – Every transaction follows the fetch, decode, and execute cycle, which slows down under heavy load.
- **High CPI** – Higher CPI means more cycles per instruction, resulting in slower transaction processing.
- **Architectural Enhancements** – Cache memory, pipelining, branch prediction, and multi-core processors reduce CPI and boost CPU performance.

How Can We Solve This Approach:



3. An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.

Understanding of the Problem Context:

The 8085-based temperature control system receives sensor data and controls actuators. Incorrect modes may cause the 8085 to read data from the wrong register or memory location, resulting in incorrect temperature readings.

Applications:

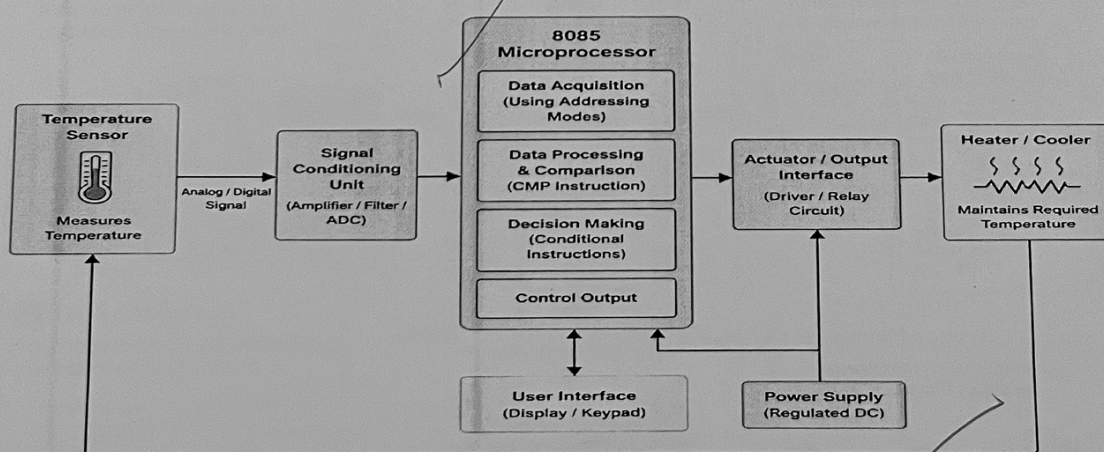
Temperature Sensor: Continuously measures the furnace temperature and sends the reading to the 8085 microprocessor.

Correct Addressing: The 8085 uses Direct or Register Indirect Addressing to read the correct sensor data.

Temperature Comparison: The CMP instruction compares the sensor value with the preset temperature limit.

Actuator Control: Based on the result, the 8085 turns the heater ON/OFF, ensuring accurate and reliable temperature control.

How can we solve this Problem:



1. **Modern Microcontroller:** Better speed, memory, and peripherals.
2. **Accurate Sensors:** Use digital sensors with noise protection.
3. **Error Detection:** Use range checking and data validation.

4. Safety System: Add watchdog timer and emergency shutdown.

4. A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.

Problem:

Segmented Memory: 8086 uses CS, DS, and SS for instructions, data, and stack.

Incorrect Data Access: Wrong segment or offset can access incorrect data.

Stack Overflow: Excessive PUSH/CALL operations can exceed stack space.

Solution: Use correct segment registers, offsets, addressing modes, and SP for reliable execution.

Applications:

Industrial Automation: 8086 can control machines and production equipment in factories.

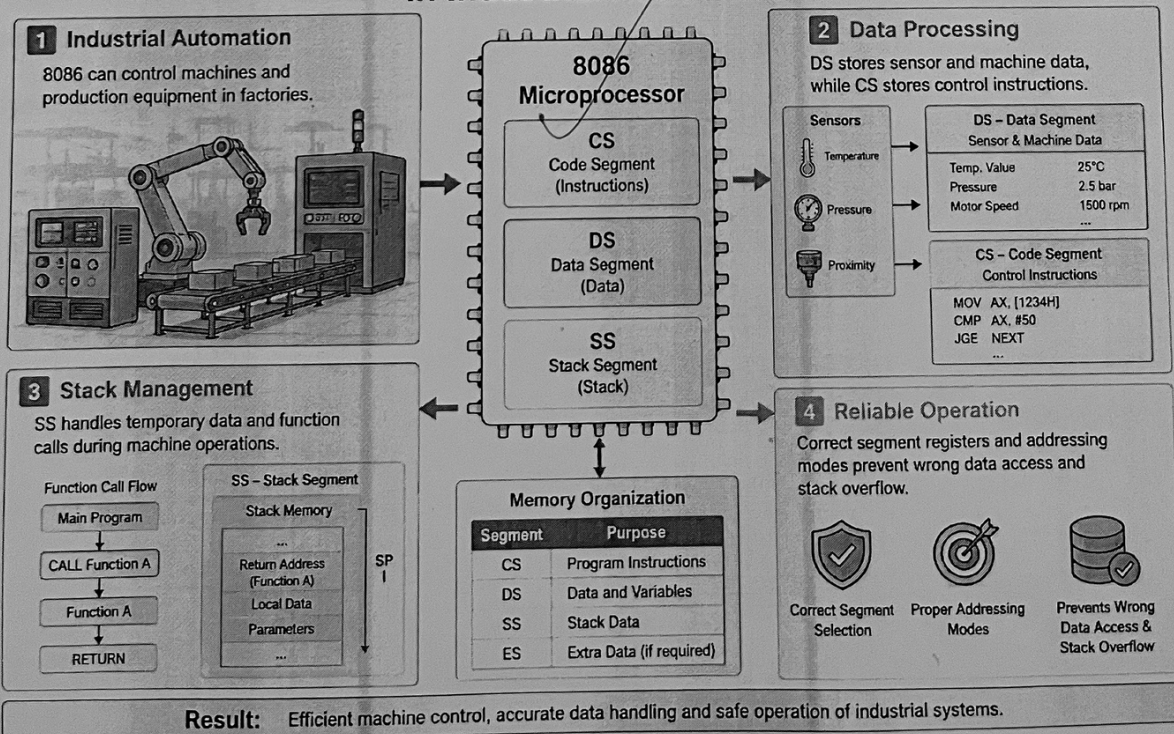
Data Processing: DS stores sensor and machine data, while CS stores control instructions.

Stack Management: SS handles temporary data and function calls during machine operations.

Reliable Operation: Correct segment registers and addressing modes prevent wrong data access and stack overflow.

How can we solve this Problem:

Real-Time Application of 8086 Microprocessor in Industrial Automation



5.

A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.

Problem:

Hexadecimal is used to represent data packets in a short and readable form, while the CPU processes them in binary. Incorrect conversion between hexadecimal and binary can produce wrong data values or instructions. This may cause incorrect CPU execution, communication errors, and system failures. Therefore, accurate conversion, CPU testing, and benchmarking are important for reliable real-time systems.

Applications:

Network Communication: Hexadecimal packets are used in routers and network devices for data transmission and debugging.

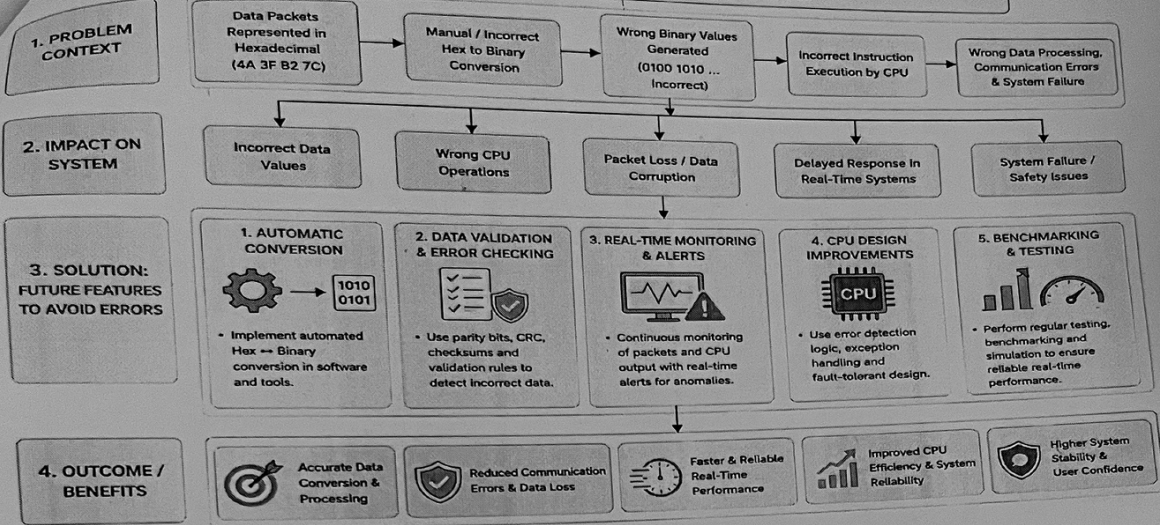
Embedded Systems: Hexadecimal values represent sensor data, memory addresses, and control instructions.

Automotive Systems: Used in ECUs to process vehicle sensor and control data.

Real-Time Systems: Accurate Hexadecimal ↔ Binary conversion ensures correct and fast CPU processing.

How can we solve this Problem:

SOLUTION APPROACH TO AVOID HEX ↔ BINARY CONVERSION ERRORS IN DATA COMMUNICATION SYSTEMS



MARKS ALLOCATION

Criteria	Understanding of Problem Context (20%)	Application of Concepts (20%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (20%)	
Q1	3	5	3	4	2	17
Q2	3	5	3	4	2	17
Q3	4	4	4	3	1.5	16.5
Q4	4	4	4	4	1.5	17.5
Q5	3	3	3	4	2	15

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements ⁴	Good understanding with minor gaps ³	Basic understanding; some requirements unclear ²	Poor understanding or misinterpretation of the problem ¹
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems ⁵	Applies relevant concepts correctly with minor errors ⁴	Applies basic concepts with limited accuracy ³	Fails to apply appropriate concepts ¹⁻²
Problem-Solving Approach	20	Logical, wellstructured, and efficient approach ⁴	Mostly logical approach with minor gaps ³	Basic approach with some inconsistencies ²	Illogical or unclear approach ¹
Accuracy of Solution	20	Completely correct solution with proper justification ⁴	Mostly correct with minor errors ³	Partially correct solution ²	Incorrect or incomplete solution ¹
Clarity	10	Clear, wellorganized, and properly explained solution ²	Understandable with minor clarity issues ^{1.5}	Limited clarity and explanation ¹	Poorly presented and difficult to understand ⁰

83